RENSEE GAJIPARA

Customer Churn Prediction

Multi-Algorithm Classification Showdown: Summary Report

1. Business Problem & Dataset Summary

A telecom operator (comparable to Jio or Airtel) loses paying subscribers every month and cannot tell in advance who is about to leave. The retention team has a limited daily call capacity (about 500 customers per day), so it needs a model that ranks customers by churn risk and a clear explanation of what drives that risk. The task is framed as **binary classification** of the target column Churn (Yes = left, No = stayed).

The project uses the **IBM Telco Customer Churn** dataset: **7,043 customers and 21 columns** covering demographics (gender, senior citizen, partner, dependents), account details (tenure, contract, payment method, paperless billing), services (phone, internet, six add-ons) and billing (MonthlyCharges, TotalCharges). **1,869 customers (26.5%) churned**, giving a retained-to-churned ratio of about 2.8 : 1. This means a model that always predicts "No Churn" would score about 73.5% accuracy while catching zero churners, so accuracy alone is misleading. The metrics that matter are Recall, F1 and AUC-ROC for the churn class.

The business cost is asymmetric. A **False Negative** (a churner predicted to stay) loses the customer's whole remaining lifetime value with no chance to react. A **False Positive** only wastes one retention call or a small discount. Recall on the churn class is therefore the primary metric, balanced against precision so the retention team is not flooded with false alarms.

2. Preprocessing & Class Imbalance Strategy

2.1 Cleaning and feature engineering

- **Cleaning:** customerID was dropped (unique identifier, no predictive value). TotalCharges loaded as text because 11 rows were blank (new customers with zero tenure); it was converted to numeric and filled with the median.
- **Engineered features:** tenure_group (New 0-12, Mid 13-36, Senior 37-60, Loyal 61+ months, ordinal-encoded 0-3), num_services (count of the six add-ons) and AutoPay (1 if the payment method is automatic; 43.5% of customers).
- **Encoding:** Yes/No columns mapped to 1/0; InternetService, Contract and PaymentMethod one-hot encoded, giving 23 model features.
- **Scaling:** StandardScaler on tenure, MonthlyCharges, TotalCharges and num_services. KNN and SVM are distance-based and need it; the same scaled matrix was given to all four models so the comparison is fair.

2.2 Avoiding data leakage

Row-wise steps (cleaning, feature engineering, fixed Yes/No mappings) were wrapped in one prepare() function that the Streamlit app reuses. The data was then split **80/20 with stratification (random_state = 42)**, giving 5,634 training and 1,409 test rows with a 26.5% churn rate in both. The scaler and one-hot encoder were fitted on the training set only.

2.3 Class imbalance: SMOTE vs class_weight

The training set held 4,139 retained and 1,495 churned customers. **SMOTE was applied to the training set only**, creating 2,644 synthetic churners so both classes reached 4,139 rows. The test set stayed untouched (1,035 retained, 374 churned) so the reported metrics reflect real, imbalanced data. As a check, the best class_weight-capable model (Decision Tree) was trained both ways. SMOTE gave slightly better Recall

(0.7995 vs 0.7888) and F1 (0.627 vs 0.620). The gap is small, so both approaches are valid; SMOTE was kept because it also helps KNN and Naive Bayes, which do not support class_weight. One side effect: Naive Bayes learned class priors of [0.5, 0.5] rather than the real [0.735, 0.265], because it saw the resampled data.

3. Recommended Model & Metric Evidence

Four classifiers were tuned and evaluated on the same held-out test set (churn class metrics): KNN (k tuned 1 to 15, best k = 15), Gaussian Naive Bayes, SVM with RBF kernel (C tuned by 3-fold CV, best C = 0.1) and Decision Tree (max_depth tuned 3 to None, best depth = 6).

Model	Accuracy	Precision	Recall	F1	AUC-ROC	Train time (s)
Decision Tree (depth 6)	**0.747**	**0.516**	**0.799**	**0.627**	**0.828**	**0.04**
SVM (C = 0.1)	0.745	0.513	0.767	0.615	0.826	10.20
Naive Bayes	0.724	0.488	0.810	0.609	0.817	0.007
KNN (k = 15)	0.722	0.485	0.799	0.604	0.817	0.005

Recommendation: deploy the Decision Tree (max_depth = 6). The reasons, with evidence:

- **Best overall balance:** highest Accuracy (0.747), Precision (0.516), F1 (0.627) and AUC-ROC (0.828) of the four models.
- **Recall is effectively tied for the lead:** 0.799 versus 0.810 for Naive Bayes, a difference of only 4 churners (299 vs 303 caught), while Naive Bayes raises more false alarms.
- **Interpretable:** the root split is Contract = Month-to-month, and the retention team can read the tree as business rules. A black-box RBF SVM cannot give that explanation.
- **Fast:** it trains in about 0.04 s versus about 10 s for the SVM, so it can be refreshed monthly and score customers in real time.
- **Ranking for the call list:** contacting the top 25% highest-risk customers (352 people) catches 212 churners (56.7%) with 60% precision, more than double the 26.5% base rate. The SVM is marginally ahead (215 churners, 57.5%), but a one-point gain does not justify losing interpretability and speed.

At the default threshold the Decision Tree catches 299 of 374 churners (Recall 0.80) and has Precision 0.52 on the churn class, i.e. roughly one in two flagged customers is a real churner.

4. Top 3 Churn Signals & Recommended Actions

The three signals below come from both the EDA and the Decision Tree feature importances (Contract_Month-to-month 58.9%, tenure 12.6%, fibre-optic 9.2%, MonthlyCharges 7.9%).

Churn signal	Evidence	Action for the retention team
1. Month-to-month contract	42.7% churn vs 11.3% (1-year) and 2.8% (2-year) customers. About 15x higher than 2-year customers. 55% of the base is month-to-month.	Offer a discount, bonus data or a free add-on to move month-to-month users onto 1- or 2-year plans, starting with the highest-risk scores.
2. Short tenure (first 12 months)	Churn is 47.4% in the first year, 28.7% at 13-24 months and 9.5% after four years. Correlation of tenure with churn is -0.35, the strongest numeric signal.	Run an onboarding programme: a check-in call in the first 30-60 days, a welcome offer and early issue resolution.

Churn signal	Evidence	Action for the retention team
3. Fibre-optic plans with high monthly bills	41.9% of fibre users churn vs 19.0% on DSL. Churners pay a median 79.65 USD vs 64.43 USD for retained customers.	Review fibre pricing and service quality; bundle OnlineSecurity or TechSupport; push AutoPay with a small incentive.

These signals compound. Month-to-month customers with above-median MonthlyCharges and tenure of 12 months or less are only **812 customers (11.5% of the base)**, yet **70% of them churn**, against 20.9% for everyone else. This segment should be the first target of any campaign.

5. What I Would Do Next

Error analysis shows the Decision Tree misses 75 of 374 churners (20%). These "quiet churners" average 26 months of tenure (vs 14.1 for caught churners), pay 63.2 USD per month (close to the 61.0 of retained customers) and only 53% are on month-to-month contracts (vs 88% of all churners). Contract, tenure and price alone cannot flag them. The next steps address this:

- **More data (highest priority):** add behavioural signals such as support complaints, usage drops, network quality, competitor offers and contract-end dates, plus several months of history so the model can learn from change over time.
- **Better models before deep learning:** try Random Forest, XGBoost or LightGBM with proper probability calibration and cost-based threshold tuning. On a 7,043-row tabular dataset, deep learning is unlikely to beat gradient boosting, so it is a lower priority.
- **Real-time scoring API:** wrap churn_model.pkl in a FastAPI service (the Streamlit app already shows the workflow) so the CRM can request a churn probability on demand and trigger retention offers automatically.
- **Monitoring and measurement:** track recall and drift monthly, retrain on fresh data, and run an A/B test of retention offers to measure real saved revenue, not just model metrics.

6. Theory Covered in This Project (Topic-wise)

Topic	Explanation
Churn, CAC and CLV	Churn is a subscriber leaving the service. Customer Acquisition Cost (CAC) is what it costs to win a new customer; Customer Lifetime Value (CLV) is the revenue a customer brings over their stay. Replacing a customer usually costs more than keeping one, so a churn model helps spend a limited retention budget where it matters.
Exploratory Data Analysis	Univariate analysis (count plots, histograms) shows the shape of single variables; bivariate analysis (churn rate by contract, tenure bucket, boxplots, correlation heatmap) shows relationships with the target. Multicollinearity appears when features move together (tenure and TotalCharges correlate at 0.83).
Data cleaning and feature engineering	Type conversion and imputation (median for TotalCharges) fix bad values. Engineered features (tenure_group, num_services, AutoPay) encode domain knowledge so models can use it directly.
Encoding and scaling	Binary mapping and one-hot encoding turn categories into numbers without implying a false order; label/ordinal encoding is used where order exists (tenure_group). StandardScaler (mean 0, std 1) is essential for distance-based models (KNN, SVM) and harmless for trees.
Train-test split and data leakage	A stratified split keeps the churn ratio equal in both sets. Any step that learns from data (scaler, encoder, SMOTE) is fitted on the training set only, otherwise test information leaks into training and the scores look better than they really are.

Topic	Explanation
Class imbalance, SMOTE and class_weight	When one class is rare, accuracy hides poor minority-class performance. SMOTE creates synthetic minority samples by interpolating between neighbouring churners; class_weight="balanced" instead makes errors on the rare class cost more. Both are applied to training data only.
K-Nearest Neighbours	A lazy learner that classifies a point by majority vote of its k closest training points. Small k over-fits to noise; large k smooths the boundary. It needs scaled features and gets slow on large data.
Gaussian Naive Bayes	Applies Bayes' theorem with the "naive" assumption that features are independent given the class, modelling each numeric feature as a Gaussian. It is very fast and often ranks well, but the independence assumption is violated here, so probabilities are poorly calibrated.
Support Vector Machine	Finds the maximum-margin boundary; the RBF kernel handles non-linear patterns. C controls the margin: a small C gives a softer margin and generalises better, a large C over-fits (best C was 0.1). It is accurate but slow and not interpretable.
Decision Tree	Splits data repeatedly on the feature that best reduces Gini impurity, producing human-readable rules and feature importances. max_depth controls over-fitting: an unrestricted tree memorised the SMOTE data (CV F1 0.523 vs 0.624 at depth 6).
Hyperparameter tuning and cross-validation	Parameters (k, C, max_depth) were selected on validation data or by k-fold cross-validation scored with F1, never on the test set, so the final test metrics remain an honest estimate.
Evaluation metrics	The confusion matrix (TP, TN, FP, FN) leads to Precision = TP/(TP+FP), Recall = TP/(TP+FN) and F1 (their harmonic mean). Recall is prioritised because a missed churner costs more than a wasted call. Precision is preferred when contacting each customer is expensive, such as a costly discount.
ROC curve and AUC	The ROC curve plots true positive rate against false positive rate across all thresholds; AUC summarises ranking quality (0.5 is random, 1.0 is perfect). Here all models land between 0.817 and 0.828, so recall, interpretability and speed decide the winner. Lowering the decision threshold trades more false alarms for more caught churners.
Error analysis and interpretability	Profiling False Negatives reveals what the model cannot see. Feature importance and the tree's root split translate the model into business language for the retention team.
Pipelines and deployment	A scikit-learn Pipeline bundles encoding, scaling and the model into one object saved with pickle/joblib (churn_model.pkl), so raw customer data can be scored the same way as in training. A Streamlit app (app.py) wraps it as an interactive churn predictor with an adjustable threshold and suggested retention actions.